第 3 章　标记、运算符和表达式

3.1.1 字符集

- ASCII：128个字符 。
- Latin-1：256个字符 。
- 16位Unicode字符集：Java平台内部使用，现行的Java在读标准的ASCII或Latin-1文件同时，将其转化为Unicode文件。
- 转义序列：'\udddd'，d是十六进制数字0-9和a-f（或A-F）,16^4=2^16。序列开头可有多个u。

  例如System.out.println('\u0022')将输出一个双引号

3.1.2 标识符

- 任何一个变量、常量、方法、对象和类都要有一个名字标志它的存在，这个名字就是标识符。Java对标识符的规定：
  - 由字母、数字、下划线、美元符号($)组合而成；
  - 以字母、下划线或美元符号开头；
  - 大小写敏感；
  - 标识符大小写敏感、长度可以任意、关键字（P39）不能作为标识符，null、true和false不是关键字，但也不能作为标识符。

3.2 类型及其字面值

- Java的每种类型都有字面值(literal)，它是相应类型的常值的书面表示 。
- 对象引用：java中唯一的文字对象引用是null，null不具有任何类型，甚至Object。
- 布尔值：true, false（0和1不是）。

整型数：整型常量可用十进制、八进制或十六进制表示。

八进制常量必须以0开头，如0110表示一个八进制数，它等于十进制数72（64+8）；

十六进制数则以0x或0X开头，如0x110表示—个十六进制数，它等于十进制数272（256+16）；

任何其它数被认为是十进制数。

整数型常量默认为32位二进制数，如要用64位表示，则在后面加英文字母l或L表示长整型。

如果把一个int的文字赋给short或byte，并且其值在short或byte的有效范围内，那么这个int的文字当作

浮点数后面跟f或F表示单精度float类型常量，跟d或D表示双精度double类型常量，缺省时是double类型常量。

一个double常量即使其值在float的有效范围内，也不能将其直接赋给一个float变量;

如果要用double常量给一个float变量赋值，必须先转换成为float。

字符文字是用单引号括起来的一个字符，以反斜杠(\)开头，将其后的字符转变为另外含义。

字符串文字是由双引号括起来的一串字符，其中允许出现所有字符常量中有效的转义序列。字符串中不允许插入换行符，而只能使用转义字符\n来代替。

```
class My {
  public static void main(String[] args) {
    boolean x1=true;x1=false;
    //x1=1;x1=2;
    short x2=300;
    //byte x3=300;int x4=65540L;
    int x5=(int)65540L;double x6=15.0f; //float x7=15.0;
    System.out.print('\n');System.out.print("\n");
    System.out.print("abc\nde");
  }
}
```

- 包装类: 每种基本类型都对应在java.lang包中一个包装类(wrapper class)：Boolean，Character，Byte，Short，Integer，Long，Float，Double。这些类中也定义了一些有用的常量和方法，如常量MIN_VALUE和MAX_VALUE。 例

```
class MyType {
  static void main(String[] args) {
    System.out.println(Integer.MIN_VALUE);
```

```
    int x; x=Integer.parseInt("100");
    System.out.println(x);
    Integer y=new Integer(100); //x=y;
    System.out.println(y+"ok"); //100ok
    int y1=y.intValue();long y2=y.longValue();
    float y3=y.floatValue();double y4=y.doubleValue();
    System.out.println(y1+","+y2+","+y3+","+y4);
    Object o;  //o=x;
    o=y;System.out.println(o);//100}
}
```

- 每一种类型都关联一个Class类的对象。可用这个Class对象对类型命名，格式为“类型名.class”。例

```
class Test {
  public static void main(String[] args) {
    Class r=String.class;
    System.out.println(r);//java.lang.String
    Class r1=boolean.class;
    System.out.println(r1);// boolean
    Object r2=int.class;
    System.out.println(r2);}}//int
```

3.3 变量

- 一个变量是一个存储实体；可把一个值赋给它而持有该值。
- 变量包括域(field)、局部变量和形参。一个变量说明要描述其标识符(名字)、类型及其它属性，其中类型确定了值的种类及可支持的行为。
- final变量必须在使用之前进行初始化。
- 可以只说明一个final域或局部变量，而推迟其初始化，见P43例。
- 类成员的说明可以同时有多个修饰符，修饰符的先后次序是无关的，建议按如下顺序：访问控制修饰符，接着static，随后synchronized，最后final。

- 局部变量可说明在一个块中的任意位置，可以是基本类型或引用类型。一个局部变量必须在使用之前指定其值，因为局部变量没有缺省值。
- 形参是说明在方法、构造器或catch子句中的形式参量。
- 形参不能被显式初始化，在方法或构造器调用时被隐式初始化为实参，或当捕捉到一个异常时用该异常对象引用来初始化形参。
- 形参和局部变量唯一可用的修饰符是final。

- 对象域或数组成员在该对象或数组用new创建时被初始化；类中static变量在任何用于该类的代码运行之前被初始化。
- 把局部变量和形参说明为final是为了方便局部内含类或匿名内含类访问。
- 类中未初始化的域将自动设置为缺省值，缺省值取决于该域的类型。

3.4 数组变量

- 数组的元素可以是基本类型，也可以是对象引用（包括对其它数组的引用）。
- 数组对象的长度在创建时固定下来，不可以再更改。
- 尽管数组是一种类型，但不能再扩展。
- int[] x和int x[]是等价的，前者更常用。
- 多维数组：数组的数组。
- 创建多维数组时，数组的第一维（最左边）必须指定，并且如果某一维已被指定，则其左边所有维必须指定，其他维可暂时不指定，但必须在后面的代码中指定。

```
class ArrayDemo {
  public static void main(String[] args) {
    int[][] x=new int[2][];
    x[0]=new int[2];//x[0]为int[]类型引用
    x[1]=new int[3];//x[1]为int[]类型引用
    for (int i=0;i<x.length;i++) {
      for (int j=0;j<x[i].length;j++) {
        x[i][j]=j;
      }
    }
```

```
        for (int i=0;i<x.length;i++) {
            for (int j=0;j<x[i].length;j++) {
                System.out.print(x[i][j]+",");}
            System.out.println();
        }
    }
}
```

0,1,

0,1,2,

- 空数组与null有很大区别，返回数组的方法在没有结果时通常返回空数组，而不是null。
- 在数组创建时，每个元素被设置为其缺省值。对于数值类型，值为0；char类型为\u0000；boolean类型为false；对象引用为null。
- 数组的初始化，见P46代码。
- 数组的多态性，见图3-1。

3.5 严格和非严格的浮点运算

- 严格浮点运算在所有Java虚拟机上都得到精确相等的结果。
- 非严格浮点运算速度相对较快。
- 用关键字strictfp来说明严格浮点运算, strictfp可修饰方法、类或接口。
- 严格浮点运算不能被继承。
- 常量表达式中的浮点运算总是严格的。

3.6 表达式

- 隐式类型转换
  - 基本数据类型的隐式转换：任一数值可以直接赋给一个其类型支持更大范围值的数值变量，例int有效的地方也可以使用char，float数可以赋给double变量；
  - 引用转换：可以在需要某超类类型引用的地方使用其子类的对象引用。null对象引用可以赋给任何类型的引用，包括数组引用和字符串引用。

- 显式类型转换和instanceof(3.6.4)
  - 有些类型转换是不允许的，例boolean型不能转换为int型；
  - 如果超类类型的引用实际指向一个子类类型的对象，则可以把超类用显式转换为该子类类型，例

```
class SuperClass {
  int x=1;
}
```

```
class SubClass extends SuperClass {
  int y=2;
  public static void main(String[] args) {
    SuperClass rSuper=new SubClass();
    //System.out.println(rSuper.y);
    SubClass rSub;
    if (rSuper instanceof SubClass) {// rSuper引用对象
      rSub=(SubClass) rSuper;
      System.out.println(rSub.y);
    }
    //System.out.println(rSub.y);}}
```

– null不是任何类型的实例，因此对任何Type而言：null instanceof Type 总是false。

- 字符串转换：在连接串时，在+的某一边若不是String，java将其转换为一个String。这种转换对所有基本类型都有定义，对于对象，这种转换是通过调用它的toString方法实现的。将null转换为String，结果为"null"。例

```
System.out.println("abc"+null);//abcnull
System.out.println("abc"+false);//abcfalse
```

3.7 访问成员

- “对象引用.静态成员名”，该引用说明类型决定访问的是哪一个类的静态成员，并非对象的实际类型决定。即使对象引用为null，也不妨碍对静态成员的访问。例

```
class A {static int x=1;}
class B extends A {static int x=2;}
class Test {
  public static void main(String[] args) {
     A r=null;
     System.out.println(r.x);
     B r1=new B();
     System.out.println(r1.x);
     r=r1;
     System.out.println(r.x);    }}
```

第 4 章　控制流

- 条件语句：if-else和switch-case-default
- 循环语句：while、do-while和for
- 跳转语句：break、continue和return
- 与异常有关的控制语句：throw、try-catch-finally

- **if-else:**

if (*boolean-expression*)

statement1

else

statement2

- **while:**

while(*boolean-expression*)

statement

- **do-while:**

do

statement

while(*boolean-expression*)

- **for:**

for(*init-expr; boolean-expression; incr-expr*)

statement

以上所有*boolean-expression*取值只能是boolean型的。

- **switch-case-default**

switch (*expression*){

case *n1*: *statements1*

case *n2*: *statements2*

…

default: *statements*

}

- switch-case-default（续）
  - switch表达式的类型必须是char、byte、short或int
  - case标号应该是整型常量；
  - 每个case值都必须是唯一的，且至多有一个default标号；
  - 一旦转到某个case标号后的语句开始执行，就一直执行，直到遇到break语句或switch语句结束，否则就可能转入下面的多个case标号；
  - 用break、return或throw来终止switch中的最后一组语句。

- **switch-case-default例**

```
class Test {
  static void method(int n) {
    switch(2*n) {
      case 2: System.out.println(1);break;
      case 4: System.out.println(2);return;
      default: System.out.println("end"); }}
  public static void main(String[] args) {
    method(1);method(5);
  }
}
```

- **break**
  - **break语句不仅可用于switch，还可从分程序(块)中退出，break语句有两种形式；**
  - **无标号的break中止最内层的switch、for、while、do-while，而且只能出现在这些语句中；**
  - **带标号的break直接退出标号所标注的语句，见代码；**

```
public boolean workOnFlag(float flag) {
    int x, y;
    boolean found = false;
```

```
search:
    for (x = 0; x < Matrix.length; x++) {
        for (y = 0; y < Matrix[x].length; y++) {
            if (Matrix[x][y] == flag) {
                found = true;
                break search; }}}
    if (!found)
        return false;
    // do some stuff with flagged value at Matrix[x][y]
    return true;
  }
```

- **continue**
  - 只能用在循环语句中
  - 在while和do-while中，continue跳到重新计算循环控制表达式
  - 无标号的continue跳到最内层循环体的末尾
  - 带标号的continue则跳到标号所指定外层循环的末尾，执行下一次外层循环，其标号必须指定一个循环语句，见代码。

```
static void doubleUp(int[][] matrix){
   int order = matrix.length;
column:
   for (int i = 0; i < order; i++){
   for (int j = 0; j < order; j++){
   matrix[i][j] = matrix[j][i] = matrix[i][j]*2;
   if (i == j)    continue column;}}}
```

```
static void doubleUp(int[][] matrix){
   int order = matrix.length;
   for (int i = 0; i < order; i++)
      for (int j = 0; j <= i; j++)
      matrix[i][j] = matrix[j][i] = matrix[i][j]*2;
}
```

- **return**
  - 终止方法执行并返回到调用代码；
  - 如果该方法没有返回值，就只需要一个简单形式： return；如果它是方法中最后一条语句，就可省略；
  - 如果方法有返回类型，那么return语句必须包含一个表达式，该表达式的值应能赋给该方法的返回类型变量；
  - return也可退出构造器和静态初始化代码，这两种情形都没有返回值，故只能用简单的return；
  - return可从嵌套的循环中直接返回，而无需标号语句或break语句，见代码。

```
public boolean workOnFlag(float flag) {
    for (int y = 0; y < Matrix.length; y++)
        for (int x = 0; x < Matrix[y].length; x++)
         if (Matrix[y][x] == flag){
      //do some stuff with flagged value at Matrix[y][x]
       return true;
    }
    return false;
}
```